

將 Cloud Spanner 連接至 GKE Autopilot

來源：<https://codelabs.developers.google.com/codelabs/cloud-spanner-gke-autopilot?hl=zh-tw>

步驟數：10

在本程式碼研究室中，您會將在 GKE Autopilot 上執行的範例服務連結至 Cloud Spanner。

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 佈建基礎架構](#)
4. [4. 建立資料庫結構定義](#)
5. [5. 部署後端服務](#)
6. [6. 部署工作負載](#)
7. [7. 開始玩遊戲](#)
8. [8. 查看 GKE 和 Spanner 用量](#)
9. [9. 正在清除所用資源](#)
10. [10. 恭喜！](#)

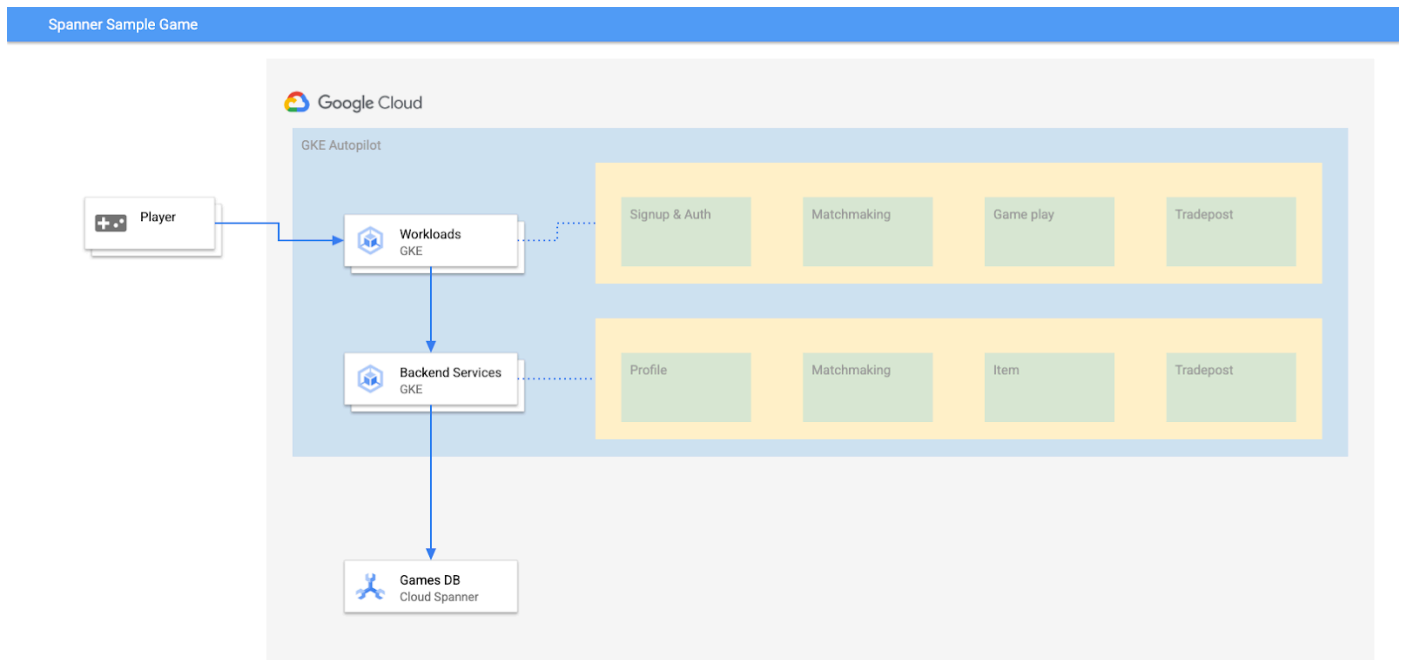
步驟 1 · 約 2 分鐘

1. 簡介

Cloud Spanner 是全代管的關聯式資料庫服務，可水平擴充並在全球各地部署，提供 ACID 交易和 SQL 語意，同時維持效能和高可用性。

GKE Autopilot 是 GKE 的作業模式，可管理叢集設定，包括節點、資源調度、安全性和其他預先設定，以遵循最佳做法。舉例來說，GKE Autopilot 可啟用 **Workload Identity** 來管理服務權限。

本實驗室的目標是逐步說明如何將 GKE Autopilot 上執行的多項後端服務，連線至 Cloud Spanner 資料庫。



在本實驗室中，您會先設定專案並啟動 Cloud Shell。接著使用 Terraform 部署基礎架構。

完成後，您將與 Cloud Build 和 Cloud Deploy 互動，對 Games 資料庫執行初始結構定義遷移作業、部署後端服務，然後部署工作負載。

本程式碼研究室中的服務與「[Cloud Spanner 遊戲開發入門](#)」程式碼研究室相同。您不必完成該程式碼研究室，也能在 GKE 上執行服務並連線至 Spanner。但如果您想進一步瞭解在 Spanner 上運作的這些服務，請參閱這篇文章。

工作負載和後端服務執行完畢後，您就可以開始產生負載，並觀察服務如何協同運作。

最後，您將清除在本實驗室中建立的資源。

建構項目

本實驗室的學習內容包括：

- 使用 Terraform 佈建基礎架構
- 使用 Cloud Build 中的結構定義遷移程序建立資料庫結構定義
- 部署四項 Golang 後端服務，利用 Workload Identity 連線至 Cloud Spanner
- 部署四項工作負載服務，用於模擬後端服務的負載。

課程內容

- 如何使用 Terraform 佈建 GKE Autopilot、Cloud Spanner 和 Cloud Deploy 管道

- Workload Identity 如何讓 GKE 上的服務模擬服務帳戶，以存取 IAM 權限來使用 Cloud Spanner
- 如何使用 [Locust.io](#) 在 GKE 和 Cloud Spanner 上產生類似於實際環境的負載

軟硬體需求

- 已連結至帳單帳戶的 Google Cloud 專案。
 - 網路瀏覽器，例如 [Chrome](#) 或 [Firefox](#)。
-

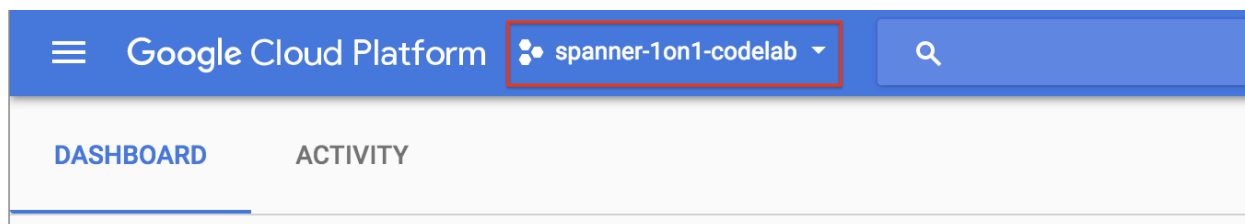
2. 設定和需求

建立專案

警告：本程式碼研究室會變更專案。強烈建議您為本程式碼研究室設定專案。

如果您沒有 Google 帳戶 (Gmail 或 Google 應用程式)，請先[建立帳戶](#)。登入 Google Cloud Platform 主控台 (console.cloud.google.com)，然後建立新專案。

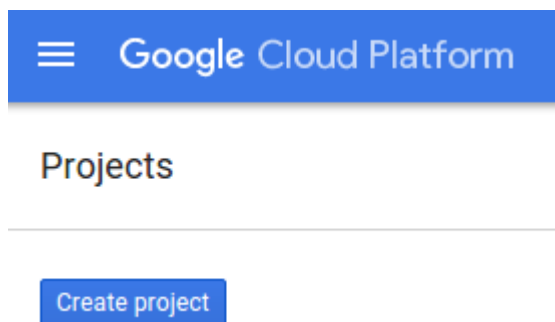
如果您已有專案，請按一下主控台左上方的專案選取下拉式選單：



然後在隨即顯示的對話方塊中，按一下「NEW PROJECT」(新專案) 按鈕，即可建立新專案：



如果您還沒有專案，應該會看到如下對話方塊，請建立第一個專案：



在隨後的專案建立對話方塊中，您可以輸入新專案的詳細資料：

New Project

Project name ?

spanner-1on1-codelab

Your project ID will be spanner-1on1-codelab ? [Edit](#)

Billing account ?

Choose account

Create

Cancel

請記住專案 ID，所有 Google Cloud 專案的專案 ID 都是不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。本程式碼研究室稍後會將其稱為 `PROJECT_ID`。

接下來，如果尚未啟用，請在開發人員控制台中[啟用帳單](#)，以便使用 Google Cloud 資源，並[啟用 Cloud Spanner API](#)。

Billing

This project has no billing account

This project is not linked to a billing account

[Link a billing account](#)

or

[Manage billing accounts](#)

完成本程式碼研究室的費用不應超過數美元，但如果您決定使用更多資源，或是將資源繼續執行 (請參閱本文件結尾的「清除」一節)，則可能會增加費用。Google Cloud Spanner 定價說明文件請參閱[這裡](#)，GKE Autopilot 說明文件請參閱[這裡](#)。

Google Cloud Platform 新使用者享有價值 **\$300 美元的免費試用期**，因此本程式碼研究室應完全免費。

設定 Cloud Shell

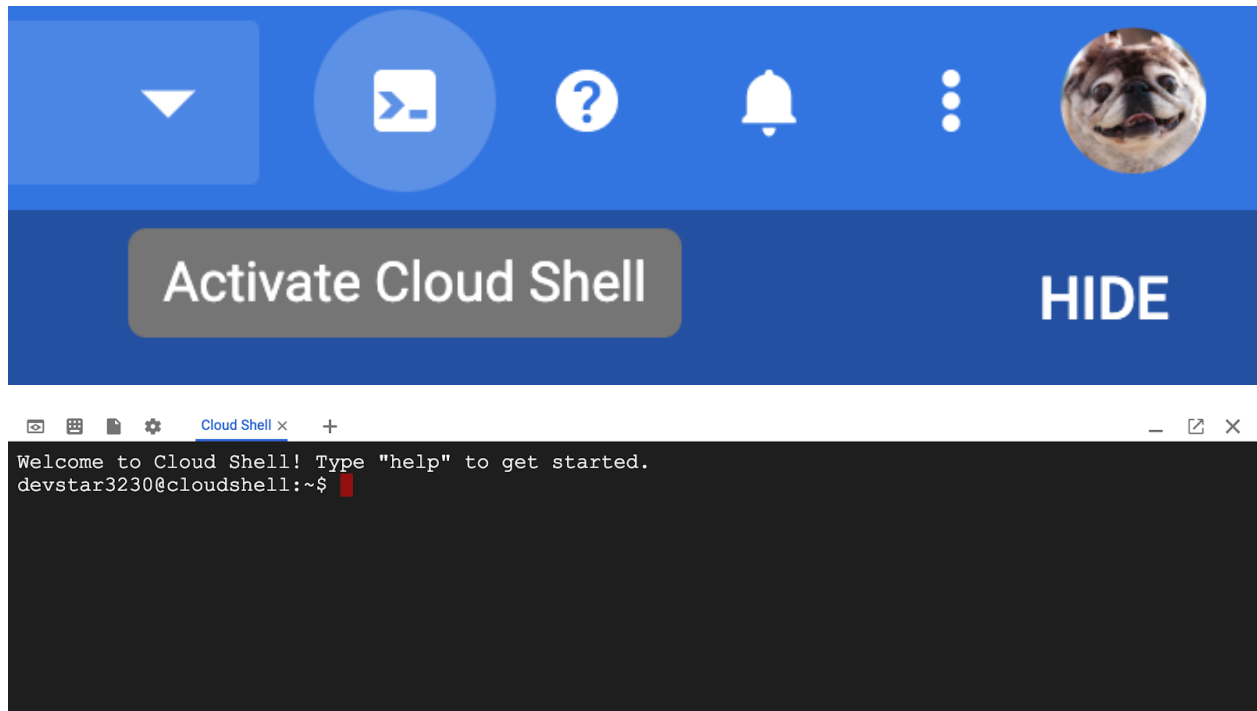
雖然可以透過筆電遠端操作 Google Cloud 和 Spanner，但在本程式碼研究室中，我們將使用 [Google Cloud Shell](#)，這是在雲端執行的指令列環境。

這部以 Debian 為基礎的虛擬機器，搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。也就是說，您只需要瀏覽器 (Chromebook 也可以) 就能完成本程式碼研究室。

1. 如要從 Cloud Shell 啟動 Cloud Shell，只要按一下「啟用 Cloud Shell」圖示



，系統就會佈建並連線至環境，這項作業只需要幾分鐘。



連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID `PROJECT_ID`。

```
gcloud auth list
```

指令輸出

```
Credentialed accounts:
- <myaccount>@<mydomain>.com (active)
```

注意：Cloud Shell 已預先安裝 gcloud 指令列工具。詳情請參閱 [gcloud 指令列工具總覽](#)。

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如果專案未設定，請發出下列指令：

```
gcloud config set project <PROJECT_ID>
```

在尋找「`PROJECT_ID`」嗎？請檢查您在設定步驟中使用的 ID，或在 Cloud 控制台資訊主頁中尋找：

 Google Cloud Platform 

DASHBOARD

ACTIVITY

RECOMMENDATIONS

 **Project info** 

Project name

my-project

Project ID

coral-bucksaw-280805

Project number

58646390136

[ADD PEOPLE TO THIS PROJECT](#)

[→ Go to project settings](#)

Cloud Shell 也會預設設定部分環境變數，這些變數在您執行後續指令時可能很有用。

```
echo $GOOGLE_CLOUD_PROJECT
```

指令輸出

```
<PROJECT_ID>
```

注意：如果在自己的機器上執行 gcloud，各個工作階段都會保留這些設定。但在 Cloud Shell 中，每次啟動新工作階段或重新連線時，都必須重新設定。

下載程式碼

在 Cloud Shell 中，您可以下載本實驗室的程式碼：

```
git clone https://github.com/cloudspannerecosystem/spanner-gaming-sample.git
```

指令輸出

```
Cloning into 'spanner-gaming-sample'...
*snip*
```

本程式碼研究室是以 [v0.1.3](#) 版本為基礎，因此請檢查該標記：

```
cd spanner-gaming-sample
git fetch --all --tags

# Check out v0.1.3 release
git checkout tags/v0.1.3 -b v0.1.3-branch
```

指令輸出

```
Switched to a new branch 'v0.1.3-branch'
```

現在，請將目前的工作目錄設為 DEMO_HOME 環境變數。這樣在完成程式碼研究室的不同部分時，就能更輕鬆地瀏覽。

```
export DEMO_HOME=$(pwd)
```

摘要

在這個步驟中，您已設定新專案、啟動 Cloud Shell，並下載本實驗室的程式碼。

下一步

接著，您將使用 Terraform 佈建基礎架構。

3. 佈建基礎架構

總覽

專案準備就緒後，就可以開始執行基礎架構。包括虛擬私有雲網路、Cloud Spanner、GKE Autopilot、Artifact Registry (用於儲存將在 GKE 上執行的映像檔)、後端服務和工作負載的 Cloud Deploy 管道，以及最後的服務帳戶和 IAM 權限，以便使用這些服務。

這是一大筆金額。但幸好，[Terraform](#) 可以簡化設定程序。Terraform 是一種「基礎架構即程式碼」工具，可讓我們在一系列「.tf」檔案中指定這個專案所需項目。這項功能可簡化基礎架構的佈建作業。

完成本程式碼研究室不需熟悉 Terraform。但如要瞭解接下來幾個步驟的用途，可以查看 [infrastructure](#) 目錄中這些檔案建立的所有內容：

- vpc.tf
- backend_gke.tf
- spanner.tf
- artifact_registry.tf
- pipelines.tf
- iam.tf

設定 Terraform

在 Cloud Shell 中，您將切換至 `infrastructure` 目錄並初始化 Terraform：

```
cd $DEMO_HOME/infrastructure
terraform init
```

指令輸出

```
Initializing the backend...

Initializing provider plugins...
*snip*
Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

接著，複製 `terraform.tfvars.sample` 並修改專案值，即可設定 Terraform。其他變數也可以變更，但只有專案必須變更，才能與您的環境搭配使用。

```
cp terraform.tfvars.sample terraform.tfvars
# edit gcp_project using the project environment variable
sed -i "s/PROJECT/$GOOGLE_CLOUD_PROJECT/" terraform.tfvars
```

注意：使用預設變數時，基礎架構會部署在 *us-central1* 區域。

佈建基礎架構

警告：這個步驟會變更專案。如果您使用現有專案，可能會導致意想不到的後果。強烈建議您為本程式碼研究室設定專案。

現在可以佈建基礎架構了！

```
terraform apply
# review the list of things to be created
# type 'yes' when asked
```

指令輸出

```
Plan: 46 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

google_project_service.project["container.googleapis.com"]: Creating...
*snip*
Apply complete! Resources: 46 added, 0 changed, 0 destroyed.
```

注意：基礎架構需要幾分鐘才能設定完畢。

注意：您可能會收到有關 Kubernetes `default_secret_name` 已淘汰的附註。您可以放心忽略這項警告。

查看建立的內容

如要驗證建立的項目，請在 Cloud 控制台中查看產品。

注意：下方資源可能要等到 `terraform apply` 步驟完成後才會顯示。

Cloud Spanner

首先，請前往漢堡選單，然後點選 `Spanner`，檢查 Cloud Spanner。您可能需要點選「查看更多產品」，才能在清單中找到該產品。

系統會將您帶往 Spanner 執行個體清單。按一下執行個體，即可查看資料庫。如下所示：

The screenshot shows the Google Cloud console interface for the Cloud Spanner service. The left sidebar contains navigation options under 'INSTANCE' (Overview, Import/Export, Backup/restore) and 'OBSERVABILITY' (System insights, Query insights, Lock insights, Transaction insights). The main content area is titled 'Overview' for the 'sample-instance'. It displays the instance name 'gaming spanner instance', ID 'sample-instance', and configuration 'us-central1 (Iowa)'. Below this, a table shows key metrics: Compute capacity (100 processing units, 0 nodes), CPU utilisation (11.99%), Operations (Read: -/s, Write: -/s), Throughput (Read: -/s, Write: -/s), and Total datab storage (0 B / 410 G). At the bottom, a 'Databases' section shows a table with one database, 'sample-game', using Google Standard SQL, with 11.99% CPU utilisation, 0 B size, and a 1-hour version retention period.

Compute capacity	CPU utilisation (mean)	Operations	Throughput	Total datab storage
Processing units: 100 Nodes: 0	11.99%	Read: -/s Write: -/s	Read: -/s Write: -/s	0 B / 410 G

Name	Dialect	CPU utilisation	Size	Version retention period
sample-game	Google Standard SQL	11.99%	0 B	1 hour

注意：您會看到這是以 100 個處理單元 (100 PU) 執行的細微執行個體，並採用 us-central 區域設定。

GKE Autopilot

接著，前往漢堡選單並點選 `Kubernetes Engine => Clusters`，即可查看 GKE。您會看到以 Autopilot 模式執行的 `sample-games-gke` 叢集。

The screenshot shows the Google Cloud console interface for the Kubernetes Engine service. The left sidebar contains navigation options under 'Kubernetes Engine' (Clusters, Workloads, Services & Ingress, Applications, and Settings and Configuration). The main content area is titled 'Kubernetes cl...' and shows a table of clusters. The 'sample-games-gke' cluster is highlighted, showing it is in the 'us-central1' location and running in 'Autopilot' mode. The table also shows 0 nodes, 0 vCPUs, and 0 GB of memory.

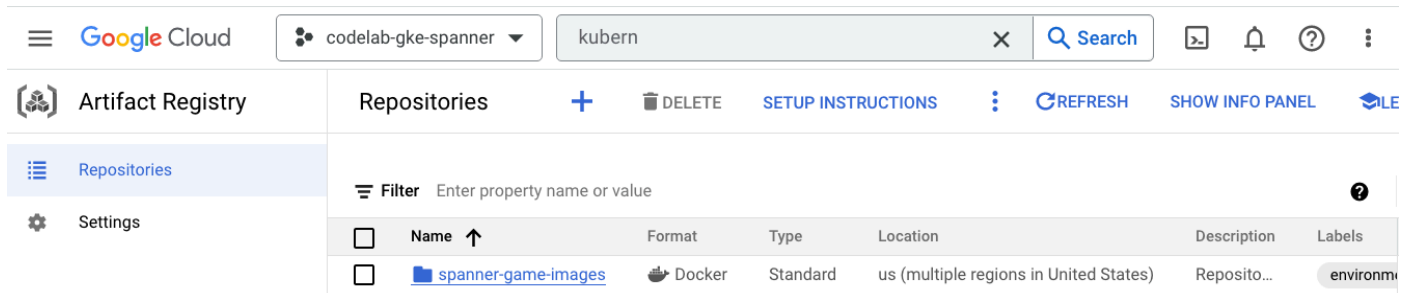
Status	Name	Location	Mode	Number of nodes	Total vCPUs	Total memory
✓	sample-games-gke	us-central1	Autopilot	0	0	0 GB

注意：請注意，GKE 叢集也位於美國中部區域。應用程式必須靠近資料庫，才能減少要求延遲時間。

Artifact Registry

現在，請查看圖片的儲存位置。因此請點選漢堡選單，然後找出 `Artifact Registry=>Repositories`。Artifact Registry 位於選單的 CI/CD 專區。

您會看到名為 `spanner-game-images` 的 Docker 登錄檔。目前這個值會是空白。



The screenshot shows the Google Cloud Artifact Registry interface. The left sidebar has 'Artifact Registry' and 'Repositories' selected. The main area shows a table of repositories. The first repository is 'spanner-game-images' with a Docker format and Standard type.

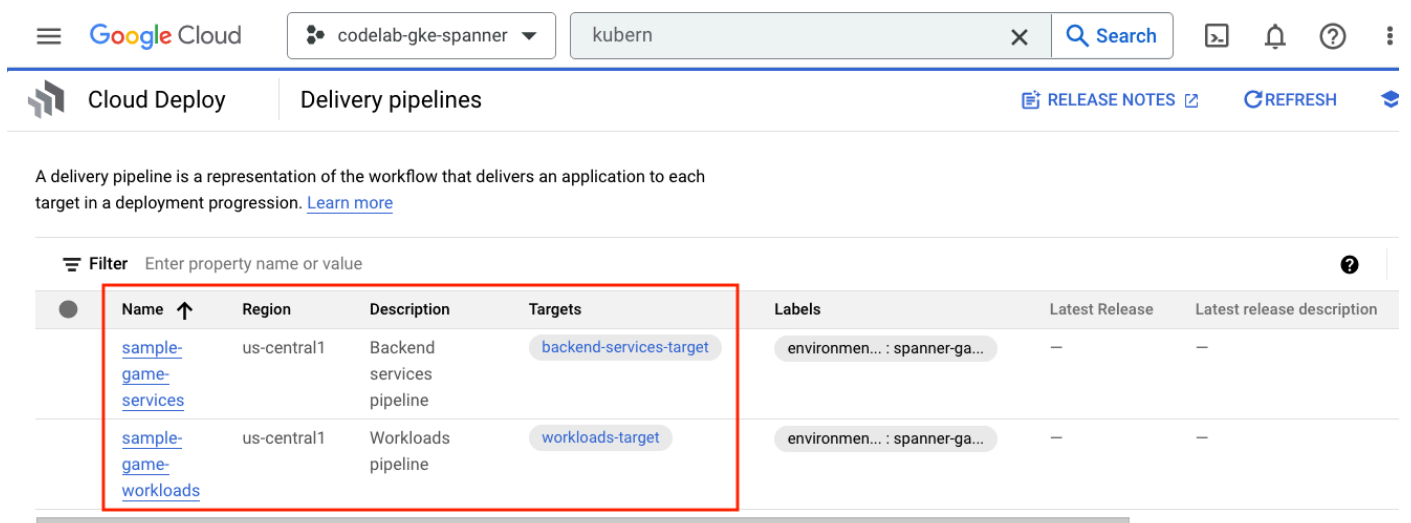
Name	Format	Type	Location	Description	Labels
spanner-game-images	Docker	Standard	us (multiple regions in United States)	Repository...	environment

Cloud Deploy

Cloud Deploy 是建立管道的位置，因此 Cloud Build 可以提供建構映像檔的步驟，然後將映像檔部署至 GKE 叢集。

前往漢堡選單，找出 `Cloud Deploy`，這個圖示也會出現在選單的 CI/CD 部分。

您會發現這裡有兩條管道：一條適用於後端服務，另一條適用於工作負載。兩者都會將映像檔部署至相同的 GKE 叢集，但這樣可以區隔部署作業。



The screenshot shows the Google Cloud Cloud Deploy interface. The left sidebar has 'Cloud Deploy' and 'Delivery pipelines' selected. The main area shows a table of delivery pipelines. Two pipelines are listed: 'sample-game-services' and 'sample-game-workloads'.

Name	Region	Description	Targets	Labels	Latest Release	Latest release description
sample-game-services	us-central1	Backend services pipeline	backend-services-target	environment: spanner-ga...	—	—
sample-game-workloads	us-central1	Workloads pipeline	workloads-target	environment: spanner-ga...	—	—

IAM

最後，請查看 Cloud 控制台的 IAM 頁面，確認已建立的服務帳戶。前往漢堡選單並找到 `IAM and Admin=>Service accounts`。如下所示：

Google Cloud

codelab-gke-spanner

iam

Search

IAM and admin

IAM

Identity and organisation

Policy troubleshooter

Policy analyser

Organisation policies

Service accounts

Workload Identity Federat...

Labels

Tags

Settings

Privacy and security

Identity-Aware Proxy

Roles

Audit logs

Manage resources

Service accounts

+ CREATE SERVICE ACCOUNT

DELETE

HELP ASSISTANT

Service accounts for project 'codelab-gke-spanner'

A service account represents a Google Cloud service identity, such as code running on Compute Engine VMs, App Engine apps or systems running outside Google. [Learn more about service accounts.](#)

Organisation policies can be used to secure service accounts and block risky service account features, such as automatic IAM Grants, key creation/upload or the creation of service accounts entirely. [Learn more about service account organisation policies.](#)

Filter

Enter property name or value

☐	Email	Status	Name ↑	Description	Key ID
☐	cloudbuild-cicd@codelab-gke-spanner.iam.gserviceaccount.com	✓	Cloud Build - CI/CD service account		No keys
☐	compute@developer.gserviceaccount.com	✓	Compute Engine default service account		No keys
☐	item-app@codelab-gke-spanner.iam.gserviceaccount.com	✓	Service account for the 'item-app' backend service		No keys
☐	matchmaking-app@codelab-gke-spanner.iam.gserviceaccount.com	✓	Service account for the 'matchmaking-app' backend service		No keys

Terraform 會建立六個服務帳戶：

- 預設的運算服務帳戶。在本程式碼研究室中，我們不會使用這項功能。
- cloudbuild-cicd 帳戶用於 Cloud Build 和 Cloud Deploy 步驟。
- 四個「應用程式」帳戶，供後端服務與 Cloud Spanner 互動。

接著，您需要設定 `kubectl`，與 GKE 叢集互動。

設定 kubectl

```
# Name of GKE cluster from terraform.tfvars file
export GKE_CLUSTER=sample-game-gke

# get GKE credentials
gcloud container clusters get-credentials $GKE_CLUSTER --region us-central1

# Check that no errors occur
kubectl get serviceaccounts
```

指令輸出

```
#export GKE_CLUSTER=sample-game-gke

# gcloud container clusters get-credentials $GKE_CLUSTER --region us-central1
Fetching cluster endpoint and auth data.
kubeconfig entry generated for sample-game-gke.

# kubectl get serviceaccounts
NAME                SECRETS  AGE
default             0        37m
item-app            0        35m
matchmaking-app     0        35m
profile-app         0        35m
trade-post-app      0        35m
```

注意：重要檢查是 `kubectl` 不應提供任何驗證錯誤。

摘要

太好了！您已在虛擬私有雲中佈建 Cloud Spanner 執行個體和 GKE Autopilot 叢集，用於私人網路。

此外，系統也為後端服務和工作負載建立了兩個 Cloud Deploy 管道，以及一個 Artifact Registry 存放區，用於儲存建構的映像檔。

最後，我們建立並設定服務帳戶，以便與 Workload Identity 搭配使用，讓後端服務能使用 Cloud Spanner。

部署後端服務和工作負載後，您也可以設定 `kubectl`，在 Cloud Shell 中與 GKE 叢集互動。

下一步

您必須先定義資料庫結構定義，才能使用這些服務。您將在下一個步驟中設定這項功能。

步驟 4 · 約 15 分鐘

4. 建立資料庫結構定義

總覽

執行後端服務前，請務必確認資料庫結構定義已就位。

如果您查看示範存放區中 `$DEMO_HOME/schema/migrations` 目錄的檔案，會看到一系列定義結構定義的 `.sql` 檔案。這會模擬開發週期，在該週期中，結構定義變更會在存放區本身中追蹤，且可與應用程式的特定功能建立關聯。

在這個範例環境中，[wrench](#) 是透過 Cloud Build 套用結構定義遷移的工具。

Cloud Build

`$DEMO_HOME/schema/cloudbuild.yaml` 檔案會說明要採取的步驟：

```
serviceAccount: projects/${PROJECT_ID}/serviceAccounts/cloudbuild-
cicd@${PROJECT_ID}.iam.gserviceaccount.com
steps:
- name: gcr.io/cloud-builders/curl
  id: fetch-wrench
  args: ['-Lo', '/workspace/wrench.tar.gz',
'https://github.com/cloudspannerecosystem/wrench/releases/download/v1.4.1/wrench-1.4.1-linux-
amd64.tar.gz' ]

- name: gcr.io/cloud-builders/gcloud
  id: migrate-spanner-schema
  entrypoint: sh
  args:
  - '-xe'
  - '-c'
  - |
    tar -xzf wrench.tar.gz

    chmod +x /workspace/wrench

    # Assumes only a single spanner instance and database. Fine for this demo in a dedicated
project
    export SPANNER_PROJECT_ID=${PROJECT_ID}
    export SPANNER_INSTANCE_ID=$(gcloud spanner instances list | tail -n1 | awk '{print $1}')
    export SPANNER_DATABASE_ID=$(gcloud spanner databases list --
instance=${SPANNER_INSTANCE_ID} | tail -n1 | awk '{print $1}')

    if [ -d ./migrations ]; then
      /workspace/wrench migrate up --directory .
    else
      echo "[Error] Missing migrations directory"
    fi
  timeout: 600s
```

基本上有兩個步驟：

- 將扳手下載至 Cloud Build 工作區

- 執行扳手遷移作業

如要讓 wrench 連線至寫入端點，您需要 Spanner 專案、執行個體和資料庫環境變數。

Cloud Build 以 `cloudbuild-cicd@${PROJECT_ID}.iam.gserviceaccount.com` 服務帳戶身分執行，因此能夠進行這些變更：

```
serviceAccount: projects/${PROJECT_ID}/serviceAccounts/cloudbuild-
cicd@${PROJECT_ID}.iam.gserviceaccount.com
```

這個服務帳戶已新增 `spanner.databaseUser` 角色 (由 Terraform 新增)，可更新 DDL。

結構定義遷移

系統會根據 `$DEMO_HOME/schema/migrations` 目錄中的檔案執行五個遷移步驟。以下是 `000001.sql` 檔案範例，可建立 `players` 資料表和索引：

```
CREATE TABLE players (
  playerUUID STRING(36) NOT NULL,
  player_name STRING(64) NOT NULL,
  email STRING(MAX) NOT NULL,
  password_hash BYTES(60) NOT NULL,
  created TIMESTAMP,
  updated TIMESTAMP,
  stats JSON,
  account_balance NUMERIC NOT NULL DEFAULT (0.00),
  is_logged_in BOOL,
  last_login TIMESTAMP,
  valid_email BOOL,
  current_game STRING(36)
) PRIMARY KEY (playerUUID);

CREATE UNIQUE INDEX PlayerAuthentication ON players(email) STORING(password_hash);
CREATE UNIQUE INDEX PlayerName ON players(player_name);
CREATE INDEX PlayerGame ON players(current_game);
```

提交結構定義遷移作業

如要提交建構作業來執行結構定義遷移作業，請切換至 `schema` 目錄，然後執行下列 gcloud 指令：

```
cd $DEMO_HOME/schema
gcloud builds submit --config=cloudbuild.yaml
```

指令輸出

```
Creating temporary tarball archive of 8 file(s) totalling 11.2 KiB before compression.
Uploading tarball of [.] to [gs://(project)_cloudbuild/source/(snip).tgz]
Created
[https://cloudbuild.googleapis.com/v1/projects/(project)/locations/global/builds/7defe982-(snip)].
Logs are available at [ https://console.cloud.google.com/cloud-build/builds/7defe982-(snip)?project=(snip) ].
```

gcloud builds submit only displays logs from Cloud Storage. To view logs from Cloud Logging, run:

```
gcloud beta builds submit
```

```
ID: 7defe982-(snip)
CREATE_TIME: (created time)
DURATION: 3M11S
SOURCE: gs://(project)_cloudbuild/source/(snip).tgz
IMAGES: -
STATUS: SUCCESS
```

注意：部署結構定義遷移作業需要幾分鐘的時間

在上述輸出內容中，您會看到 `Created` 雲端建構程序的連結。點選後，系統會將您帶往 Cloud 控制台中的建構作業，方便您監控建構作業的進度，並查看作業內容。

Cloud Build

Dashboard

History

Triggers

Settings

Build details

REBUILD

COPY URL

LEARN

Successful: 7defe982

Started on 3 Apr 2023, 10:58:01

Source

gs://

Steps	Duration	BUILD LOG	EXECUTION DETAILS	BUILD ARTIFACTS
Build summary	00:03:11			
2 Steps				
0: fetch-wrench	00:00:01			
-Lo /workspace/wre...				
1: migrate-spanner-s...	00:03:00			
sh -xe -c tar -xvzf wr...				

Filter

Search all fields and values

?

↺

↻

SEVERITY	TIMESTAMP	SUMMARY
> i	2023-04-03 10:58:16.773 EDT	Step #1 - "migrate-spanner-schema": + export SP_
> i	2023-04-03 10:58:16.774 EDT	Step #1 - "migrate-spanner-schema": + gcloud sp_
> i	2023-04-03 10:58:16.774 EDT	Step #1 - "migrate-spanner-schema": + awk {prin_
> i	2023-04-03 10:58:16.776 EDT	Step #1 - "migrate-spanner-schema": + tail -n1
> i	2023-04-03 10:58:18.455 EDT	Step #1 - "migrate-spanner-schema": + export SP_
> i	2023-04-03 10:58:18.455 EDT	Step #1 - "migrate-spanner-schema": + [-d ./mi_
> i	2023-04-03 10:58:18.455 EDT	Step #1 - "migrate-spanner-schema": + /workspac_
> i	2023-04-03 10:58:27.191 EDT	Step #1 - "migrate-spanner-schema": 1/up
> i	2023-04-03 10:58:48.166 EDT	Step #1 - "migrate-spanner-schema": 2/up
> i	2023-04-03 10:58:56.941 EDT	Step #1 - "migrate-spanner-schema": 3/up
> i	2023-04-03 11:00:29.360 EDT	Step #1 - "migrate-spanner-schema": 4/up
> i	2023-04-03 11:01:12.736 EDT	Step #1 - "migrate-spanner-schema": 5/up
> i	2023-04-03 11:01:13.065 EDT	Finished Step #1 - "migrate-spanner-schema"
> i	2023-04-03 11:01:13.065 EDT	PUSH
> i	2023-04-03 11:01:13.065 EDT	DONE

Release notes

注意：遷移步驟的數量與 `$DEMO_HOME/schema/migrations` 目錄中的檔案數量相符。

摘要

在這個步驟中，您使用 Cloud Build 提交了初始結構定義遷移作業，其中套用了 5 種不同的 DDL 作業。這些作業代表新增需要變更資料庫結構定義的功能。

在一般開發情境中，您會希望結構定義變更能回溯相容於目前的應用程式，避免發生中斷。

對於不具回溯相容性的變更，建議您分階段將變更部署至應用程式和結構定義，確保不會發生中斷。

下一步

設定結構定義後，下一步就是部署後端服務！

5. 部署後端服務

總覽

本程式碼研究室的後端服務是 go lang REST API，代表四種不同的服務：

- **個人資料**：讓玩家註冊及驗證我們的範例「遊戲」。
- **配對**：與玩家資料互動，協助配對功能運作、追蹤遊戲建立的相關資訊，並在遊戲關閉時更新玩家數據。
- **項目**：讓玩家在遊戲過程中取得遊戲物品和金錢。
- **交易站**：讓玩家在交易站買賣物品



如要進一步瞭解這些服務，請參閱「[Cloud Spanner Getting Started with Games Development](#)」程式碼研究室。就我們的目的而言，我們希望這些服務在 GKE Autopilot 叢集上執行。

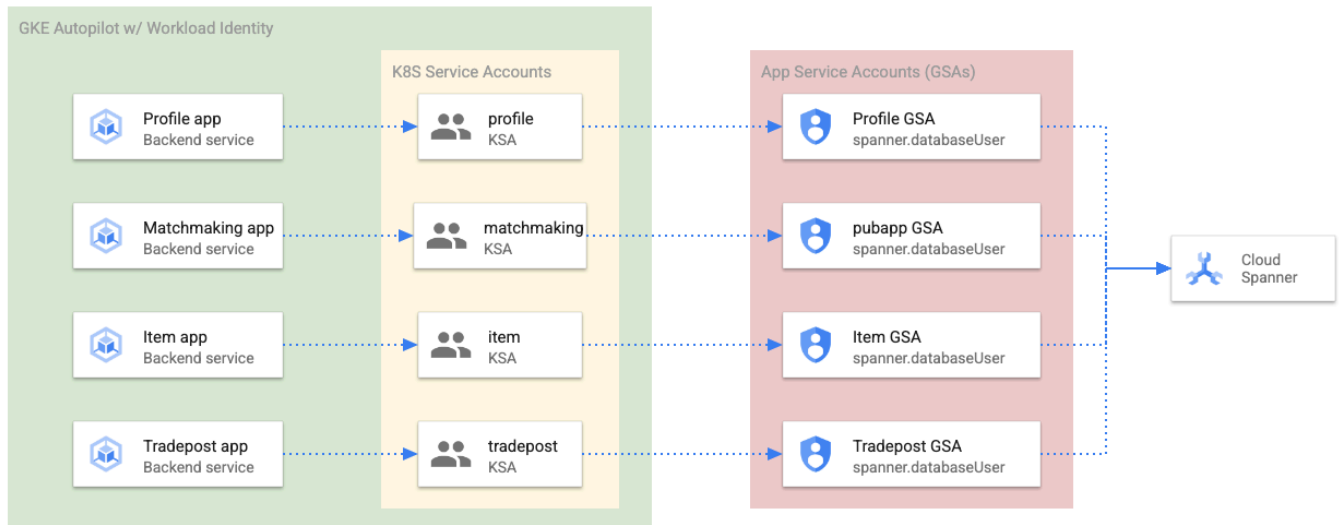
這些服務必須能夠修改 Spanner 資料。為此，每項服務都會建立服務帳戶，並授予「databaseUser」角色。

透過 Workload Identity，Kubernetes 服務帳戶可以模擬服務的 Google Cloud 服務帳戶，方法是在 Terraform 中執行下列步驟：

- 建立服務的 Google Cloud 服務帳戶 (`GSA`) 資源
- 將 `databaseUser` 角色指派給該服務帳戶
- 將 `workloadIdentityUser` 角色指派給該服務帳戶
- 建立參照 GSA 的 Kubernetes 服務帳戶 (`KSA`)

粗略的圖表如下所示：

Workload Identity Example



注意：每項服務都有自己的 GSA 和 KSA，可區隔權限並遵循「最少特殊權限」原則。

Terraform 已為您建立服務帳戶和 Kubernetes 服務帳戶。您可以使用 `kubectl` 檢查 Kubernetes 服務帳戶：

```
# kubectl get serviceaccounts
NAME                SECRETS  AGE
default             0        37m
item-app            0        35m
matchmaking-app     0        35m
profile-app         0        35m
tradepost-app       0        35m
```

建構作業的運作方式如下：

- Terraform 生成的 `$DEMO_HOME/backend_services/cloudbuild.yaml` 檔案大致如下：

```

serviceAccount: projects/${PROJECT_ID}/serviceAccounts/cloudbuild-
cicd@${PROJECT_ID}.iam.gserviceaccount.com
steps:

#
# Building of images
#
- name: gcr.io/cloud-builders/docker
  id: profile
  args: ["build", ".", "-t", "${_PROFILE_IMAGE}"]
  dir: profile
  waitFor: ['-']
- name: gcr.io/cloud-builders/docker
  id: matchmaking
  args: ["build", ".", "-t", "${_MATCHMAKING_IMAGE}"]
  dir: matchmaking
  waitFor: ['-']
- name: gcr.io/cloud-builders/docker
  id: item
  args: ["build", ".", "-t", "${_ITEM_IMAGE}"]
  dir: item
  waitFor: ['-']
- name: gcr.io/cloud-builders/docker
  id: tradepost
  args: ["build", ".", "-t", "${_TRADEPOST_IMAGE}"]
  dir: tradepost
  waitFor: ['-']

#
# Deployment
#
- name: gcr.io/google.com/cloudsdktool/cloud-sdk
  id: cloud-deploy-release
  entrypoint: gcloud
  args:
    [
      "deploy", "releases", "create", "${_REL_NAME}",
      "--delivery-pipeline", "sample-game-services",
      "--skaffold-file", "skaffold.yaml",
      "--skaffold-version", "1.39",
      "--images",
      "profile=${_PROFILE_IMAGE},matchmaking=${_MATCHMAKING_IMAGE},item=${_ITEM_IMAGE},tradepost=${_TRADEPOST_IMAGE}",
      "--region", "us-central1"
    ]

artifacts:
  images:
    - ${_REGISTRY}/profile
    - ${_REGISTRY}/matchmaking
    - ${_REGISTRY}/item
    - ${_REGISTRY}/tradepost

```

```

substitutions:
  _PROFILE_IMAGE: ${_REGISTRY}/profile:${BUILD_ID}
  _MATCHMAKING_IMAGE: ${_REGISTRY}/matchmaking:${BUILD_ID}
  _ITEM_IMAGE: ${_REGISTRY}/item:${BUILD_ID}
  _TRADEPOST_IMAGE: ${_REGISTRY}/tradepost:${BUILD_ID}
  _REGISTRY: us-docker.pkg.dev/${PROJECT_ID}/spanner-game-images
  _REL_NAME: rel-${BUILD_ID:0:8}
options:
  dynamic_substitutions: true
  machineType: E2_HIGHCPU_8
  logging: CLOUD_LOGGING_ONLY

```

- Cloud Build 指令會讀取這個檔案，並按照列出的步驟操作。首先，它會建構服務映像檔。然後執行 `gcloud deploy create` 指令。這會讀取 `$DEMO_HOME/backend_services/skaffold.yaml` 檔案，該檔案會定義每個部署檔案的位置：

```

apiVersion: skaffold/v2beta29
kind: Config
deploy:
  kubectrl:
    manifests:
      - spanner_config.yaml
      - profile/deployment.yaml
      - matchmaking/deployment.yaml
      - item/deployment.yaml
      - tradepost/deployment.yaml

```

- Cloud Deploy 會遵循各項服務 `deployment.yaml` 檔案的定義。服務的部署檔案包含建立服務的資訊，在本例中，該服務是在通訊埠 80 上執行的 clusterIP。

「**ClusterIP**」類型可防止後端服務 Pod 擁有外部 IP，因此只有能連線至內部 GKE 網路的實體，才能存取後端服務。這些服務會存取及修改 Spanner 資料，因此玩家不應直接存取。

```

apiVersion: v1
kind: Service
metadata:
  name: profile
spec:
  type: ClusterIP
  selector:
    app: profile
  ports:
    - port: 80
      targetPort: 80

```

範例：`$DEMO_HOME/backend_services/profile/deployment.yaml`

除了建立 Kubernetes 服務，Cloud Deploy 也會建立 Kubernetes 部署作業。讓我們檢查 `profile` 服務的部署部分：

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: profile
spec:
  replicas: 2 # EDIT: Number of instances of deployment
  selector:
    matchLabels:
      app: profile
  template:
    metadata:
      labels:
        app: profile
    spec:
      serviceAccountName: profile-app
      containers:
      - name: profile-service
        image: profile
        ports:
        - containerPort: 80
        envFrom:
        - configMapRef:
            name: spanner-config
        env:
        - name: SERVICE_HOST
          value: "0.0.0.0"
        - name: SERVICE_PORT
          value: "80"
      resources:
        requests:
          cpu: "1"
          memory: "1Gi"
          ephemeral-storage: "100Mi"
        limits:
          cpu: "1"
          memory: "1Gi"
          ephemeral-storage: "100Mi"
```

頂端部分提供服務的一些中繼資料。其中最重要的部分是定義這個部署作業要建立多少個副本。

```
replicas: 2 # EDIT: Number of instances of deployment
```

接著，我們會看到應用程式應由哪個服務帳戶執行，以及應使用哪個映像檔。這些項目會與透過 Terraform 建立的 Kubernetes 服務帳戶，以及在 Cloud Build 步驟中建立的映像檔相符。

```
spec:
  serviceAccountName: profile-app
  containers:
    - name: profile-service
      image: profile
```

接著，我們會指定網路和環境變數的相關資訊。

`spanner_config` 是 Kubernetes ConfigMap，用於指定應用程式連線至 Spanner 時所需的專案、執行個體和資料庫資訊。

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: spanner-config
data:
  SPANNER_PROJECT_ID: ${project_id}
  SPANNER_INSTANCE_ID: ${instance_id}
  SPANNER_DATABASE_ID: ${database_id}
```

注意： `spanner_config.yaml` 檔案是由 Terraform 根據 files/backend_services/spanner-config.yaml.tpl 產生。

```
ports:
  - containerPort: 80
envFrom:
  - configMapRef:
      name: spanner-config
env:
  - name: SERVICE_HOST
    value: "0.0.0.0"
  - name: SERVICE_PORT
    value: "80"
```

`SERVICE_HOST` 和 `SERVICE_PORT` 是服務所需的額外環境變數，可讓服務瞭解要繫結的位置。

最後一個部分會告知 GKE，要允許此部署中的每個副本使用多少資源。GKE Autopilot 也是使用這項功能，視需要擴充叢集。

```
resources:
  requests:
    cpu: "1"
    memory: "1Gi"
    ephemeral-storage: "100Mi"
  limits:
    cpu: "1"
    memory: "1Gi"
    ephemeral-storage: "100Mi"
```

掌握以上資訊後，即可部署後端服務。

部署後端服務

如前所述，部署後端服務時會使用 Cloud Build。與結構定義遷移作業相同，您可以使用 `gcloud` 指令列提交建構要求：

```
cd $DEMO_HOME/backend_services
gcloud builds submit --config=cloudbuild.yaml
```

指令輸出

```
Creating temporary tarball archive of 66 file(s) totalling 864.6 KiB before compression.
Uploading tarball of [.] to [gs://(project)_cloudbuild/source/(snip).tgz]
Created
[https://cloudbuild.googleapis.com/v1/projects/(project)/locations/global/builds/30207dd1-(snip)].
Logs are available at [ https://console.cloud.google.com/cloud-build/builds/30207dd1-(snip)?project=(snip) ].

gcloud builds submit only displays logs from Cloud Storage. To view logs from Cloud Logging,
run:
gcloud beta builds submit

ID: 30207dd1-(snip)
CREATE_TIME: (created time)
DURATION: 3M17S
SOURCE: gs://(project)_cloudbuild/source/(snip).tgz
IMAGES: us-docker.pkg.dev/(project)/spanner-game-images/profile:30207dd1-(snip) (+3 more)
STATUS: SUCCESS
```

注意：建構及部署作業需要幾分鐘才能完成。

與 `schema migration` 步驟的輸出內容不同，這個建構作業的輸出內容表示已建立一些圖片。這些構件會儲存在 Artifact Registry 存放區中。

`gcloud build` 步驟的輸出內容會包含 Cloud 控制台的連結。請參考這些範例。

注意：即使 Cloud Build 傳回 SUCCESS 訊息，部署作業仍須等到 Pod 執行完畢才會完成。

收到 Cloud Build 的成功通知後，請前往 Cloud Deploy，然後前往 `sample-game-services` 管道，監控部署進度。

Pipeline visualisation

backend-services-t...
rel-30207dd1
1 hour ago

Deployments

1 in last 30 days

Deployment frequency

Monthly averaging 0 days per month

Deployment failure rate

0% failure in last 30 days

服務部署完成後，您可以檢查 `kubectl`，查看 Pod 的狀態：

```
kubectl get pods
```

指令輸出

NAME	READY	STATUS	RESTARTS	AGE
item-6b9d5f678c-4tbk2	1/1	Running	0	83m
matchmaking-5bcf799b76-lg8zf	1/1	Running	0	80m
profile-565bbf4c65-kphdl	1/1	Running	0	83m
profile-565bbf4c65-xw74j	1/1	Running	0	83m
tradepost-68b87ccd44-gw55r	1/1	Running	0	79m

接著，請檢查服務，看看 `ClusterIP` 的運作情形：

```
kubectl get services
```

指令輸出

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
item	ClusterIP	10.172.XXX.XXX	<none>	80/TCP	84m
kubernetes	ClusterIP	10.172.XXX.XXX	<none>	443/TCP	137m
matchmaking	ClusterIP	10.172.XXX.XXX	<none>	80/TCP	84m
profile	ClusterIP	10.172.XXX.XXX	<none>	80/TCP	84m
tradepost	ClusterIP	10.172.XXX.XXX	<none>	80/TCP	84m

注意：後端服務不會指派外部 IP，以免外部存取 API。

您也可以前往 Cloud 控制台的 GKE UI，查看 `workloads`、`Services` 和 `ConfigMaps`。

工作負載

Kubernetes Engine

Clusters

Workloads

Services & Ingress

Applications

Secrets and ConfigMaps

Storage

Object Browser

Migrate to Containers

Backup for GKE

Security posture

Workloads

REFRESH

DEPLOY

DELETE

OPERATIONS

HELP ASSISTANT

LEARN

Cluster

Namespace

RESET

SAVE

Workloads are deployable units of computing that can be created and managed in a cluster.

OVERVIEWOBSERVABILITYCOST OPTIMISATION

FilterIs system object : FalseFilter workloads

	Name	Status	Type	Pods	Namespace	Cluster
☐	item	OK	Deployment	1/1	default	sample-game-gke
☐	matchmaking	OK	Deployment	1/1	default	sample-game-gke
☐	profile	OK	Deployment	2/2	default	sample-game-gke
☐	tradeport	OK	Deployment	1/1	default	sample-game-gke

服務

Kubernetes Engine

Clusters

Workloads

Services & Ingress

Applications

Secrets and ConfigMaps

Storage

Object Browser

Migrate to Containers

Backup for GKE

Security posture

Services & Ingress

REFRESH

CREATE INGRESS

DELETE

HELP ASSISTANT

LEARN

Cluster

Namespace

RESET

SAVE

SERVICESINGRESS

Services are sets of pods with a network endpoint that can be used for discovery and load balancing. Ingresses are collections of rules for routing external HTTP(S) traffic to Services.

FilterIs system object : FalseFilter services and ingresses

	Name	Status	Type	Endpoints	Pods	Namespace	Clusters
☐	item	OK	Cluster IP	10.172.	1/1	default	sample-game-gke
☐	matchmaking	OK	Cluster IP	10.172.	1/1	default	sample-game-gke
☐	profile	OK	Cluster IP	10.172.	2/2	default	sample-game-gke
☐	tradeport	OK	Cluster IP	10.172.	1/1	default	sample-game-gke

ConfigMap

Kubernetes Engine

Clusters

Workloads

Services & Ingress

Applications

Secrets and ConfigMaps

Storage

Object Browser

Migrate to Containers

Backup for GKE

Security posture

Configuration

REFRESH

DELETE

HELP ASSISTANT

LEARN

Cluster

Namespace

RESET

SAVE

Secrets are sensitive pieces of information, such as passwords, keys and tokens. ConfigMaps are designed to store information that is not sensitive, such as environment variables, command-line arguments and configuration files.

Secrets respect access control and are not visible to users without read permissions

FilterIs system object : FalseFilter secrets and config maps

	Name	Type	Namespace	Cluster
☐	kube-root-ca.crt	Config Map	kube-public	sample-game-gke
☐	kube-root-ca.crt	Config Map	default	sample-game-gke
☐	kube-root-ca.crt	Config Map	kube-node-lease	sample-game-gke
☐	spanner-config	Config Map	default	sample-game-gke

Kubernetes Engine

Clusters

Workloads

Services & Ingress

Applications

Secrets and ConfigMaps

Storage

Object Browser

Migrate to Containers

Backup for GKE

Security posture

← Config map d... REFRESH EDIT DELETE

spanner-config

DETAILS YAML

Cluster

sample-game-gke

Namespace

default

Created

3 Apr 2023, 11:24:55

Labels

No labels set

Annotations

SHOW ANNOTATIONS

Data

SPANNER_DATABASE_ID

sample-game

SPANNER_INSTANCE_ID

sample-instance

SPANNER_PROJECT_ID

code-lab-gke-spanner

摘要

在這個步驟中，您已將四項後端服務部署至 GKE Autopilot。您已順利執行 Cloud Build 步驟，並在 Cloud Deploy 和 Cloud Console 的 Kubernetes 中查看進度。

您也瞭解這些服務如何運用 Workload Identity，模擬具有適當權限的服務帳戶，以便讀取及寫入 Spanner 資料庫的資料。

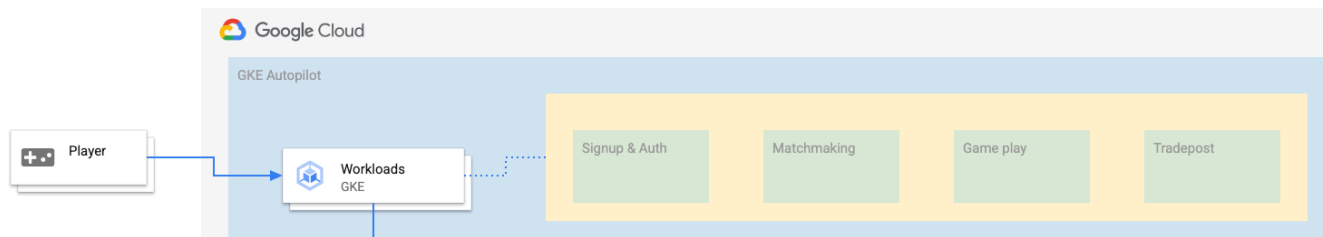
後續步驟

您會在下一節中部署工作負載。

6. 部署工作負載

總覽

後端服務已在叢集上執行，現在要部署工作負載。



工作負載可從外部存取，且本程式碼研究室的每個後端服務都有一個工作負載。

這些工作負載是以 [Locust](#) 為基礎的負載產生指令碼，可模擬這些範例服務預期的實際存取模式。

Cloud Build 程序有下列檔案：

- `$DEMO_HOME/workloads/cloudbuild.yaml` (由 Terraform 產生)
- `$DEMO_HOME/workloads/skaffold.yaml`
- 每個工作負載的 `deployment.yaml` 檔案

歡迎查看 `cloudbuild.yaml` 和 `skaffold.yaml` 檔案。這與後端服務幾乎完全相同，只是映像檔名稱和目錄不同。

工作負載 `deployment.yaml` 檔案與後端服務部署檔案略有不同。

以下是 `matchmaking-workload` 的範例：

```

apiVersion: v1
kind: Service
metadata:
  name: matchmaking-workload
spec:
  type: LoadBalancer
  selector:
    app: matchmaking-workload
  ports:
    - port: 8089
      targetPort: 8089
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: matchmaking-workload
spec:
  replicas: 1 # EDIT: Number of instances of deployment
  selector:
    matchLabels:
      app: matchmaking-workload
  template:
    metadata:
      labels:
        app: matchmaking-workload
    spec:
      serviceAccountName: default
      containers:
        - name: matchmaking-workload
          image: matchmaking-workload
          ports:
            - containerPort: 8089
          resources:
            requests:
              cpu: "500m"
              memory: "512Mi"
              ephemeral-storage: "100Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
              ephemeral-storage: "100Mi"

```

檔案頂端會定義服務。在此情況下，系統會建立 `LoadBalancer`，工作負載則會在通訊埠 `8089` 上執行。

`LoadBalancer` 會提供外部 IP，可用於連線至工作負載。

```
apiVersion: v1
kind: Service
metadata:
  name: matchmaking-workload
spec:
  type: LoadBalancer
  selector:
    app: matchmaking-workload
  ports:
    - port: 8089
      targetPort: 8089
```

部署作業區段頂端是工作負載的中繼資料。在本例中，系統只會部署一個副本：

```
replicas: 1
```

不過，容器規格有所不同。首先，我們使用的是 `default` Kubernetes 服務帳戶。由於工作負載不需要連線至 GKE 叢集上執行的後端服務以外的任何 Google Cloud 資源，因此這個帳戶沒有任何特殊權限。

另一項差異是這些工作負載不需要任何環境變數。因此部署規格會較短。

```
spec:
  serviceAccountName: default
  containers:
    - name: matchmaking-workload
      image: matchmaking-workload
  ports:
    - containerPort: 8089
```

資源設定與後端服務類似。請注意，GKE Autopilot 就是根據這項資訊，判斷滿足叢集上所有 Pod 要求所需的資源數量。

請繼續部署工作負載！

部署工作負載

與先前一樣，您可以使用 `gcloud` 指令列提交建構要求：

```
cd $DEMO_HOME/workloads
gcloud builds submit --config=cloudbuild.yaml
```

指令輸出

```
Creating temporary tarball archive of 18 file(s) totalling 26.2 KiB before compression.
Some files were not included in the source upload.
```

```
Check the gcloud log [/tmp/tmp.4Z9EqdPo6d/logs/(snip).log] to see which files and the
contents of the
default gcloudignore file used (see `gcloud topic gcloudignore` to learn
more).
```

```
Uploading tarball of [.] to [gs://(project)_cloudbuild/source/(snip).tgz]
Created
[https://cloudbuild.googleapis.com/v1/projects/(project)/locations/global/builds/(snip)].
Logs are available at [ https://console.cloud.google.com/cloud-build/builds/0daf20f6-(snip)?
project=(snip) ].
```

```
gcloud builds submit only displays logs from Cloud Storage. To view logs from Cloud Logging,
run:
gcloud beta builds submit
```

```
ID: 0daf20f6-(snip)
CREATE_TIME: (created_time)
DURATION: 1M41S
SOURCE: gs://(project)_cloudbuild/source/(snip).tgz
IMAGES: us-docker.pkg.dev/(project)/spanner-game-images/profile-workload:0daf20f6-(snip) (+4
more)
STATUS: SUCCESS
```

注意：建構及部署作業需要幾分鐘才能完成。

請務必在 Cloud 控制台中查看 Cloud Build 記錄和 Cloud Deploy 管道，瞭解狀態。工作負載的 Cloud Deploy 管道為 `sample-game-workloads`：

部署完成後，請在 Cloud Shell 中使用 `kubectl` 檢查狀態：

```
kubectl get pods
```

指令輸出

NAME	READY	STATUS	RESTARTS	AGE
game-workload-7ff44cb657-pxxq2	1/1	Running	0	12m
item-6b9d5f678c-cr29w	1/1	Running	0	9m6s
item-generator-7bb4f57cf8-5r85b	1/1	Running	0	12m
matchmaking-5bcf799b76-lg8zf	1/1	Running	0	117m
matchmaking-workload-76df69dbdf-jds9z	1/1	Running	0	12m
profile-565bbf4c65-kphdl	1/1	Running	0	121m
profile-565bbf4c65-xw74j	1/1	Running	0	121m
profile-workload-76d6db675b-kzwng	1/1	Running	0	12m
tradeport-68b87ccd44-gw55r	1/1	Running	0	116m
tradeport-workload-56c55445b5-b5822	1/1	Running	0	12m

接著检查工作負載服務，查看 `LoadBalancer` 的運作情形：

```
kubectl get services
```

指令輸出

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
game-workload	LoadBalancer	*snip*	35.XX.XX.XX	8089:32483/TCP	12m
item	ClusterIP	*snip*	<none>	80/TCP	121m
item-generator	LoadBalancer	*snip*	34.XX.XX.XX	8089:32581/TCP	12m
kubernetes	ClusterIP	*snip*	<none>	443/TCP	174m
matchmaking	ClusterIP	*snip*	<none>	80/TCP	121m
matchmaking-workload	LoadBalancer	*snip*	34.XX.XX.XX	8089:31735/TCP	12m
profile	ClusterIP	*snip*	<none>	80/TCP	121m
profile-workload	LoadBalancer	*snip*	34.XX.XX.XX	8089:32532/TCP	12m
tradeport	ClusterIP	*snip*	<none>	80/TCP	121m
tradeport-workload	LoadBalancer	*snip*	34.XX.XX.XX	8089:30002/TCP	12m

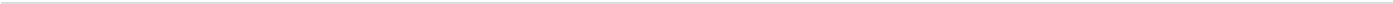
注意：與後端服務不同，工作負載會指派外部 IP 位址。

摘要

您現在已將工作負載部署至 GKE 叢集。這些工作負載不需要額外的 IAM 權限，且可透過 [LoadBalancer 服務](#)，從外部通訊埠 8089 存取。

後續步驟

後端服務和工作負載執行完畢後，就可以「玩」遊戲了！



7. 開始玩遊戲

總覽

範例「遊戲」的後端服務現在正在執行，您也可以使用工作負載產生與這些服務互動的「玩家」。

每項工作負載都會使用 [Locust](#)，模擬對服務 API 的實際負載。在這個步驟中，您會執行多項工作負載，在 GKE 叢集和 Spanner 上產生負載，並將資料儲存在 Spanner 上。

以下說明各項工作負載：

- `item-generator` 工作負載可快速產生 `game_items` 清單，列出玩家在「玩」遊戲時可取得的項目。
- `profile-workload` 會模擬玩家註冊及登入。
- `matchmaking-workload` 會模擬玩家排隊等候分配到遊戲。
- `game-workload` 會模擬玩家在遊戲過程中取得 `game_items` 和金錢。
- `trade-post-workload`：模擬玩家在交易站買賣物品。

注意：如要進一步瞭解這些 Locust 產生器如何運作，請參閱「[Cloud Spanner Getting Started with Games Development](#)」程式碼研究室。

本程式碼研究室將特別著重於執行 `item-generator` 和 `profile-workload`。

執行項目產生器

`item-generator` 會使用 `item` 後端服務端點，將 `game_items` 新增至 Spanner。`game-workload` 和 `trade-post-workload` 必須有這些項目才能正常運作。

第一步是取得 `item-generator` 服務的外部 IP。在 Cloud Shell 中執行下列指令：

```
# The external IP is the 4th column of the output
kubectl get services | grep item-generator | awk '{print $4}'
```

指令輸出

```
{ITEMGENERATOR_EXTERNAL_IP}
```

現在，開啟新的瀏覽器分頁，並指向 `http://{ITEMGENERATOR_EXTERNAL_IP}:8089`。您應該會看到類似下方的頁面：

Start new load test

Number of users (peak concurrency)

Spawn rate (users started/second)

Host (e.g. http://www.example.com)

Advanced options

Start swarming

請將 `users` 和 `spawn` 保留為預設值 1。在 `host` 中輸入 `http://item`。按一下進階選項，然後輸入 `10s` 做為執行時間。

設定應如下所示：

Start new load test

Number of users (peak concurrency)

Spawn rate (users started/second)

Host (e.g. <http://www.example.com>)

Advanced options

Run time (e.g. 20, 20s, 3m, 2h, 1h20m, 3h30m10s, etc.)

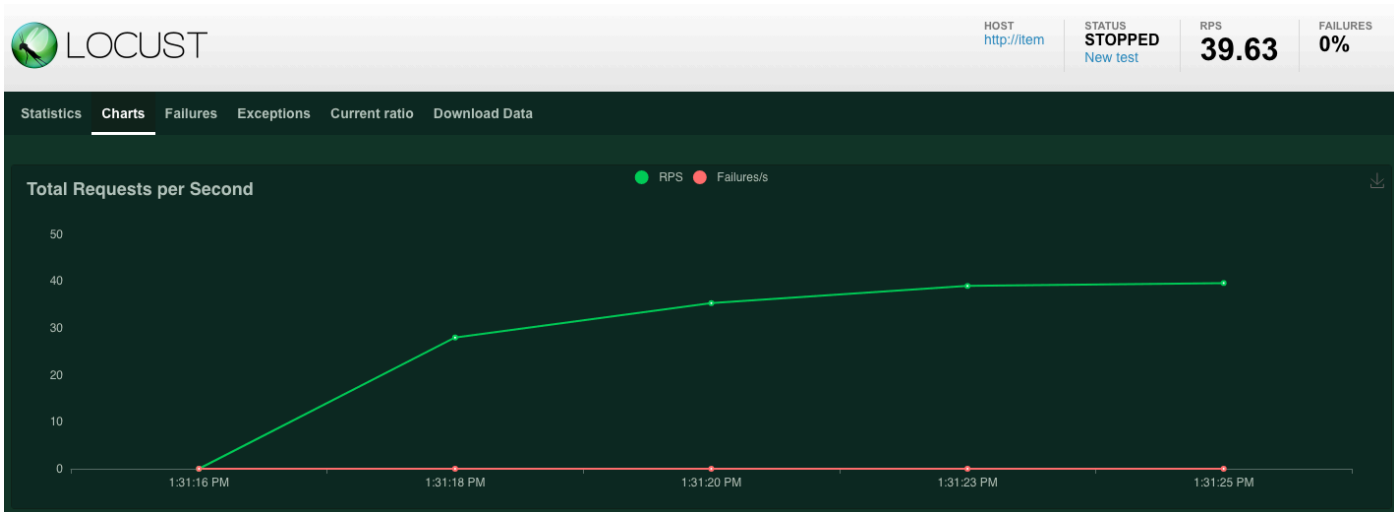
Start swarming

注意：由於工作負載和後端服務位於同一個 GKE 叢集，因此可以透過服務名稱的 [DNS 解析](#) 功能互相探索。因此您可以參照 `http://item` 主機名稱。

按一下「Start swarming」！

系統會開始顯示在 `POST /items` 端點發出的要求統計資料。10 秒後，負載就會停止。

按一下 `Charts`，即可查看這些要求的成效圖表。



現在，您要檢查資料是否已輸入 Spanner 資料庫。

如要這麼做，請按一下漢堡選單，然後前往「Spanner」。從這個頁面前往 `sample-instance` 和 `sample-database`。然後按一下「Query」。

我們要選取 `game_items` 的數量：

```
SELECT COUNT(*) FROM game_items;
```

畫面底部會顯示結果。

Spanner

All instances > INSTANCE sample-instance: Overview > GOOGLE STANDARD SQL DATABASE sample-game: Query

QUERY 1

+ NEW TAB QUERY TEMPLATES

RUN CLEAR QUERY FORMAT QUERY SHORTCUTS SQL query help

```
1 SELECT COUNT(*) FROM game_items;
```

SCHEMA RESULTS EXPLANATION

All results > SELECT COUNT(*) FROM game_items;

Query completed (3.22 ms elapsed)

Result
410

注意：遊戲項目數量可能不完全一致。沒關係！

我們不需要大量 `game_items`。但現在玩家可以取得這些物品了！

執行設定檔工作負載

`game_items` 完成種子設定後，下一個步驟是讓玩家註冊，以便暢玩遊戲。

`profile-workload` 會使用 Locust 模擬玩家建立帳戶、登入、擷取個人資料資訊及登出。這些測試都會在類似於一般生產環境的工作負載中，測試 `profile` 後端服務的端點。

如要執行這項操作，請取得 `profile-workload` 外部 IP：

```
# The external IP is the 4th column of the output
kubectl get services | grep profile-workload | awk '{print $4}'
```

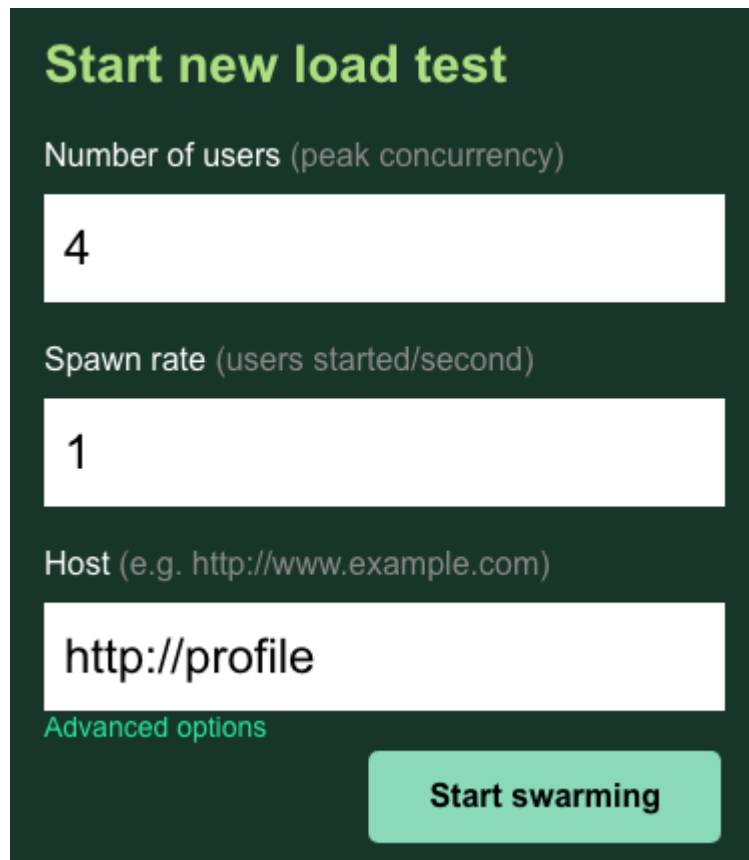
指令輸出

```
{PROFILEWORKLOAD_EXTERNAL_IP}
```

現在，開啟新的瀏覽器分頁，並指向 `http://{PROFILEWORKLOAD_EXTERNAL_IP}:8089`。您應該會看到類似先前的 Locust 頁面。

在這種情況下，您會使用 `http://profile` 做為主機。您也不會在進階選項中指定執行階段。此外，請將 `users` 指定為 4，一次模擬 4 個使用者要求。

`profile-workload` 測試應如下所示：



Start new load test

Number of users (peak concurrency)

4

Spawn rate (users started/second)

1

Host (e.g. `http://www.example.com`)

`http://profile`

[Advanced options](#)

Start swarming

按一下「Start swarming」！

和之前一樣，各種 `profile` REST 端點的統計資料會開始顯示。按一下圖表，即可查看所有項目的成效。



注意：請讓 `profile-workload` 保持執行狀態。您可以在下一節中查看 GKE 和 Cloud Spanner 的效能。

摘要

在這個步驟中，您產生了一些 `game_items`，然後使用 Cloud 控制台中的 Spanner 查詢使用者介面查詢 `game_items` 資料表。

您也允許玩家註冊遊戲，並瞭解 Locust 如何針對後端服務建立類似正式版的負載。

後續步驟

執行工作負載後，您會想檢查 GKE 叢集和 Spanner 執行個體的行為。

步驟 8 · 約 10 分鐘

8. 查看 GKE 和 Spanner 用量

設定檔服務執行完畢後，即可查看 GKE Autopilot 叢集和 Cloud Spanner 的運作情形。

檢查 GKE 叢集

前往 Kubernetes 叢集。請注意，部署工作負載和服務後，叢集現在會新增總 vCPU 和記憶體的一些詳細資料。如果叢集上沒有任何工作負載，系統就不會提供這項資訊。

Kubernetes Engine

Clusters

Workloads

Services & Ingress

Applications

Secrets and ConfigMaps

Kubernetes cl...

CREATE

DEPLOY

OPERATIONS

HELP ASSISTANT

LEARN

OVERVIEW

OBSERVABILITY

COST OPTIMISATION

Filter

Enter property name or value

Status	Name ↑	Location	Mode	Number of nodes	Total vCPUs	Total memory	N
☐	☒	sample-game-gke	us-central1	Autopilot	6	6 GB	

現在，按一下 `sample-game-gke` 叢集，然後切換至「可觀測性」分頁：

sample-game-gke

DETAILS

STORAGE

OBSERVABILITY

LOGS

ENABLE MANAGED PROMETHEUS

RECOMMENDED ALERTS

SAVE AS DASHBOARD

Overview

CPU

Memory

Kubernetes events

Application Integrations

RESET ZOOM

1 hour

6 hours

1 day

1 week

1 month

6 weeks

Custom

CPU request % used (Top 5 namespaces)

3 Apr 2023, 13:37:00

default 33.45%

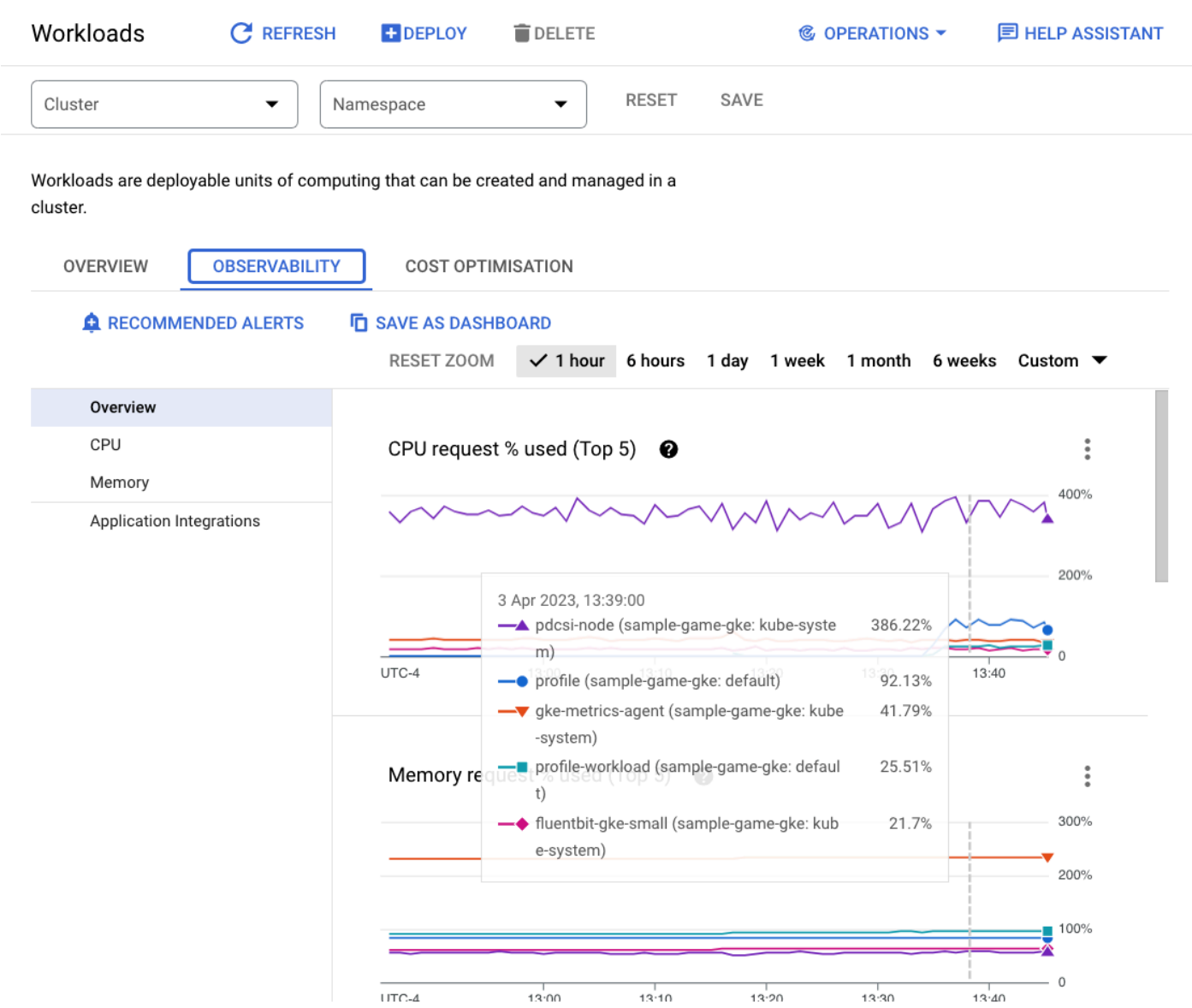
kube-system 18%

Memory request % used (Top 5 namespaces)

由於工作負載和後端服務都在 `default` 上執行，因此 `default` Kubernetes 命名空間的 CPU 使用率應已超過 `kube-system` 命名空間。如果沒有，請確認 `profile workload` 仍在執行，並稍候幾分鐘，等待圖表更新。

如要查看哪些工作負載佔用最多資源，請前往 `Workloads` 資訊主頁。

不必逐一查看每個工作負載，直接前往資訊主頁的「可觀測性」分頁即可。您應該會看到 `profile` 和 `profile-workload` CPU 使用率增加。

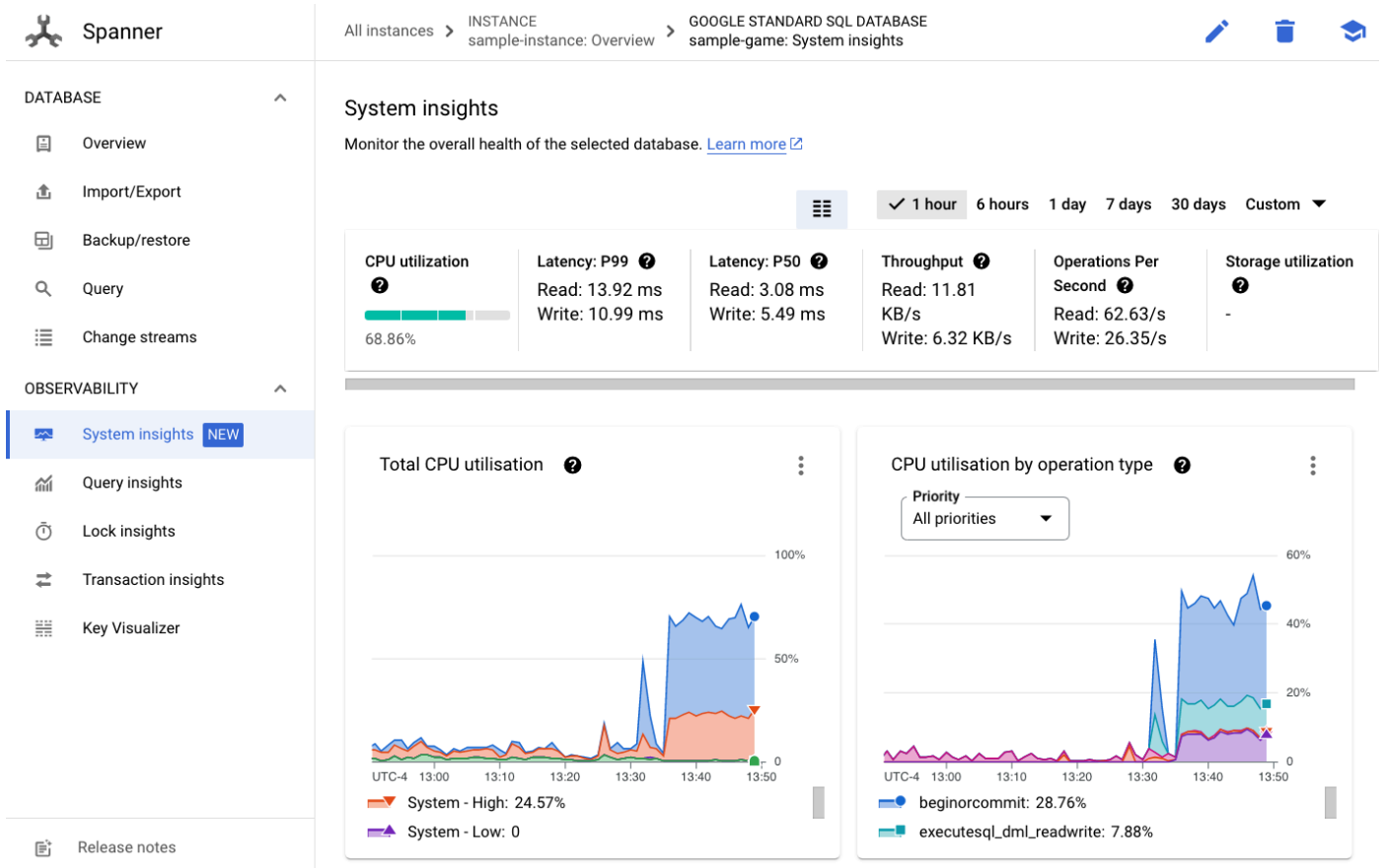


現在，請前往 Cloud Spanner 查看。

檢查 Cloud Spanner 執行個體

如要檢查 Cloud Spanner 的效能，請前往 Spanner，然後點選 `sample-instance` 執行個體和 `sample-game` 資料庫。

接著，您會在左選單中看到「系統洞察」分頁：



這裡有許多圖表可協助您瞭解 Spanner 執行個體的整體效能，包括 CPU utilization、transaction latency and locking 和 query throughput。

除了系統洞察之外，您也可以查看「可觀測性」部分的其他連結，取得查詢工作負載的詳細資訊：

- 查詢洞察功能可協助找出 Spanner 上使用最多資源的前 N 個查詢。
- 交易和鎖定深入分析有助於找出高延遲的交易。
- Key Visualizer 可協助您以視覺化方式呈現存取模式，並追蹤資料中的熱點。

摘要

在本步驟中，您已瞭解如何檢查 GKE Autopilot 和 Spanner 的基本效能指標。

舉例來說，在執行設定檔工作負載時，查詢 *players* 資料表，即可取得儲存在該處的資料相關資訊。

在繼續操作之前，請花點時間瀏覽 GKE 和 Spanner 上的可用資訊，熟悉這些服務。

後續步驟

接著，我們來清除主題！

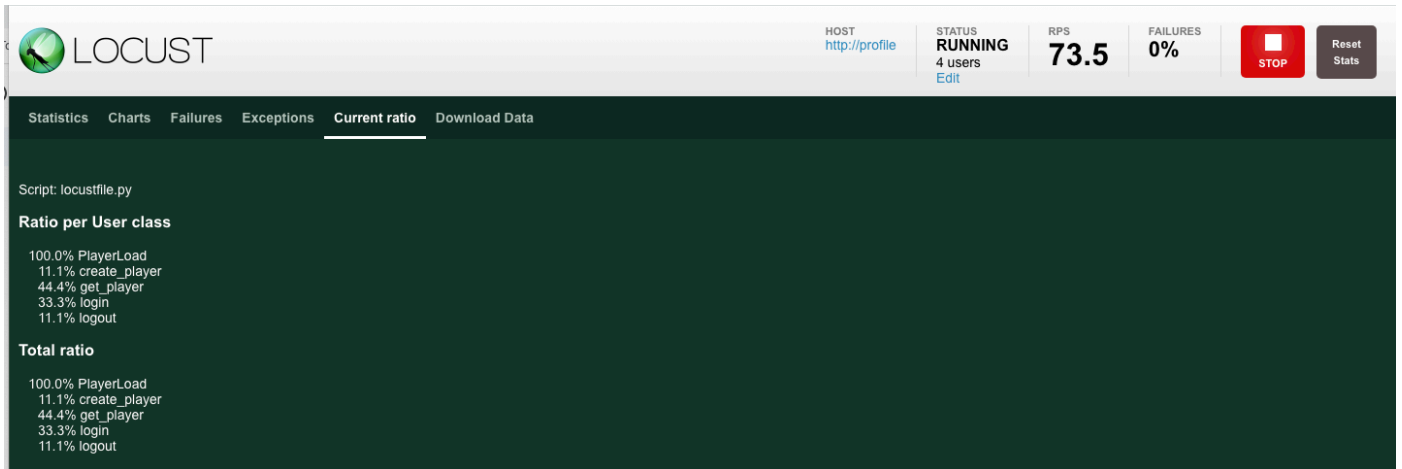
步驟 9 · 約 2 分鐘

9. 正在清除所用資源

在清除資源前，歡迎探索未涵蓋的其他工作負載。具體來說，就是 `matchmaking-workload`、`game-workload` 和 `tradepost-workload`。

「玩」完遊戲後，可以清理遊樂場。所幸這項作業相當簡單。

首先，如果 `profile-workload` 仍在瀏覽器中執行，請前往並停止執行：



對您測試過的每個工作負載執行相同操作。

接著在 Cloud Shell 中，前往基礎架構資料夾。您將使用 Terraform `destroy` 基礎架構：

```
cd $DEMO_HOME/infrastructure
terraform destroy
# type 'yes' when asked
```

指令輸出

```
Plan: 0 to add, 0 to change, 46 to destroy.

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

*snip*

Destroy complete! Resources: 46 destroyed.
```

注意：完全移除基礎架構需要幾分鐘的時間。

在 Cloud 控制台中，依序前往 `Spanner`、`Kubernetes Cluster`、`Artifact Registry`、`Cloud Deploy` 和 `IAM`，確認所有資源都已移除。

步驟 10 · 約 2 分鐘

10. 恭喜！

恭喜！您已成功在 GKE Autopilot 上部署 Go 範例應用程式，並使用 Workload Identity 將應用程式連線至 Cloud Spanner！

此外，這項基礎架構可使用 Terraform 輕鬆設定及移除，且可重複操作。

如要進一步瞭解本程式碼研究室中使用的 Google Cloud 服務，請參閱下列資源：

- [GKE Autopilot](#) 和 [工作負載身分](#)
- [Cloud Spanner](#)
- [Artifact Registry](#)
- [Cloud Build](#) 和 [Cloud Deploy](#)

後續步驟

您已瞭解 GKE Autopilot 和 Cloud Spanner 的協作方式，不妨開始建構自己的應用程式，搭配使用這些服務。
